

4.0 Java Programming Language

4.1 Introduction to Java Program

Java Output & Input Statements

COMPUTER SCIENCE II| SC025

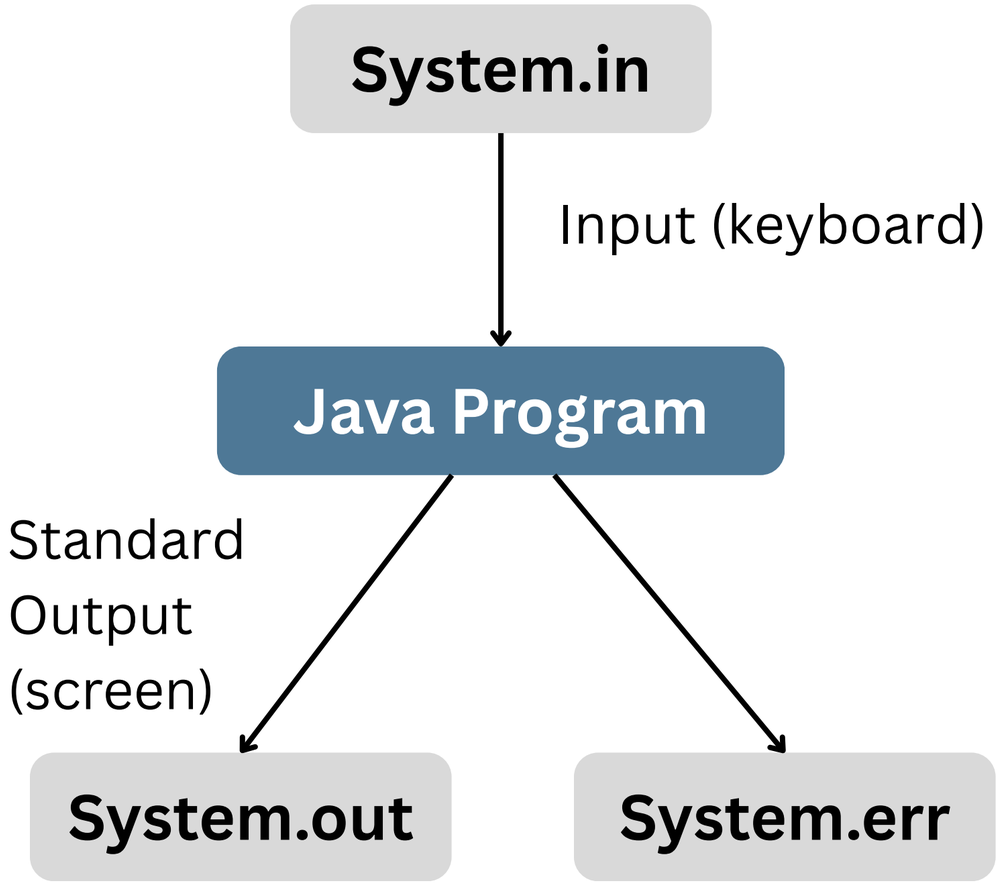
Prepared by: Ms Jasini

Learning Outcomes

4.1 Introduction to Java Program

c. Use output and input statements including output String.

Java Input & Output



Java handles standard input and output using ***System.in***, ***System.out***, and *System.err*.

- *System.in* receives input from user
- *System.out* displays normal output

*We will only be using these two input and output handling methods in this course.

Output Statements

System.out is the **standard output stream** used in Java.

We will use various methods from this stream to display output from ur program:

- ***print()***
- ***println()***
- ***printf()***

Each method will display output differently, so it is important to choose the most appropriate method you need for your program.

*This course focuses on *print()* and *println()* mainly, but it is useful to know *printf()* too.

print( ) method

The **print()** method displays the message that is in the brackets onto the output terminal.

Statement:	Output:
`System.out.print("Hello");`	`Hello`
`System.out.print("Hello");` `System.out.print("world!");`	`Helloworld!`
`System.out.print("Hello world!");`	`Hello world!`

- ***print()*** method starts printing according to the last position of the output terminal.
- whitespace must be INCLUDED in message in order for it to be printed.

println() method

The output **println()** method displays the message that is in the brackets onto the output terminal then inserts a new line.

Statement:

```
System.out.println("Hello");
System.out.println("world!");
```

Output:

```
Hello
world!
```

- ***println()*** method moves to the next line in the output terminal **AFTER** printing its message.

print() and *println()* method

Both ***print()*** and ***println()*** methods allows you to append (add) variables and messages together using the **+** symbol.

Statement:

```
String name = "John"; //String is the data type for messages
int age = 20;//int is the data type for whole numbers
System.out.println("Hello" + "World");
System.out.println("Name: " + name + " Age: " + age);
```

Output:

```
HelloWorld
Name: John Age: 20
```

printf() method

The **printf()** method prints a formatted string. It is useful for displaying specific numbers of decimal places for decimal values.

Example (double or float values):

```
System.out.printf("Cost RM%.2f, Price RM%.2f", 2, 5.0);
System.out.printf("\nPi to 3 decimal places is %.3f", 3.14159);
```

Output:

```
Cost RM2.00, Price RM5.00
Pi to 3 decimal places is 3.141
```

- *% indicates where the value should be printed*
- *.2 indicates the number of decimal places*
- *f indicates the data type (f - float)*
- Visit https://www.w3schools.com/java/ref_output_printf.asp for more info.

printf() method

Examples (varying data types):

```
String name = "Mark";
int age = 30;
System.out.printf("My name is %s and I am %d years old.", name, age);
```

```
My name is Mark and I am 30 years old.
```

```
int num1 = sc.nextInt();
double num2 = sc.nextDouble();
System.out.printf("%d x %.1f = %.1f", num1, num2, num1*num2);
```

```
1
2
1 x 2.0 = 2.0
```

Escape Characters

We can use escape characters to achieve specific results in print statements.

Escape Character	Description
\’	Single quote ‘
\”	Double quote “
\\	Backslash
\n	New Line
\t	Tab

Example

Desired Output: He said “ I don’t want that. ”

```
System.out.print(“He said “ I don’t want that. ””);
```



```
System.out.print(“He said \“ I don\’t want that. \””);
```



Desired Output: | | | |

```
System.out.print(“|\t|\t|\t|”);
```

Desired Output:
Hello
World

```
System.out.print(“Hello\nWorld”);
```

Output Statements

REMINDER

- *whitespace* (space) is NOT automatic. If you want your text spaced, YOU have to make sure there is whitespace in your text.
- for multiple spaces / neater formating use a tab instead (\t)
- use escape characters to reduce the amount of code that needs to be written.
- be aware of the line spacing difference when using *println()* and *\n escape character*.
    - *println()* - goes to new line AFTER printing text given
    - *\n* - goes to new line at the place of the escape character

Input Statements

System.in is the **standard input stream** used in Java.

Input statements are statements used to **accept and store data** into the computer.

- Data may be entered by user or initialized by programmer.
- Data will be stored in a variable (a location in the computer memory).

In this course we will focus on using the ***Scanner*** class and its methods to read various inputs of different data types.

Scanner class

The Scanner class is used to get **user input**.

- It is found in the java.util **package**.
- Import the package using *import java.util.Scanner;* at the start of your code.
- To use the Scanner class we first need to create an object from the class.

```
import java.util.Scanner; //Import Scanner class
public class UserInput{
    public static void main(String[] args){
        //Create Scanner object
        Scanner sc = new Scanner(System.in);
    }
}
```

next() method

The ***next()*** method reads text up to the first whitespace.

Example (String):

```
import java.util.Scanner;
public class UserInput{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String name = sc.next(); //Read user input
        System.out.print("Name is " + name);
    }
}
```

Output:

```
Enter name: Jasini Kalaichelvan
Name is Jasini
```

nextLine() method

The nextLine() method returns all text in a single line of input until the first line break.

Example (String):

```
import java.util.Scanner; //Import Scanner class
public class UserInput{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in); //Create Scanner object
        System.out.print("Enter name: ");
        String name = sc.nextLine(); //Read user input
        System.out.print("Name is " + name);
    }
}
```

```
Enter name: Jasini Kalaichelvan
Name is Jasini Kalaichelvan
```

Input Types

Choose and use the correct Scanner class methods based on your desired input data types.

Method	Description
`nextBoolean()`	Reads a `boolean` value from the user
`nextByte()`	Reads a `byte` value from the user
`nextDouble()`	Reads a `double` value from the user
`nextFloat()`	Reads a `float` value from the user
`nextInt()`	Reads a `int` value from the user
`nextLine()`	Reads a `String` value from the user

Full Example

```
import java.util.Scanner; //Import Scanner class
public class UserInput{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in); //Create Scanner object
        System.out.print("Enter age, weight, and marital status:");
        int age = sc.nextInt(); //Read integer user input
        double weight = sc.nextDouble(); //Read double user input
        boolean married = sc.nextBoolean(); //Read boolean user input
        System.out.println("Age: " + age);
        System.out.println(" Weight: " + weight + "kg");
        System.out.println(" Married: " + married);
    }
}
```

```
Enter age, weight, and marital status: 25
65.5
false
Age: 25 Weight: 65.5kg Married: false
```

next().charAt() method

There is no specified method to read a char input type. Instead we use the next().charAt() method to obtain a specified character in the first string read by the next() method.

```
import java.util.Scanner; //Import Scanner class
public class UserInput{
   public static void main(String[] args){
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a three letter word: ");
      char firstChar = sc.next().charAt(0);
      char secondChar = sc.next().charAt(1);
      char thirdChar = sc.next().charAt(2);
      System.out.println(firstChar);
      System.out.println(secondChar);
      System.out.println(thirdChar);
   }
}
```

Output:

```
Enter a three letter
word: THE
T
H
E
```

**The number in the bracket specifies the position. The first position starts at 0.

Input Statements

REMINDER

- It is good practice to **use an output statement** before our **input statement** to prompt the user as the scanner methods only takes in values.

Without prompt:

```
Scanner sc = new Scnner(System.in);
int age = sc.nextInt();
System.out.print("My age is " + age);
```

```
25
My age is 25
```

With prompt:

```
Scanner sc = new Scnner(System.in);
System.out.print("Enter age: ");
int age = sc.nextInt();
System.out.print("My age is " + age);
```

```
Enter age: 25
My age is 25
```